

Integracja ESP32 z AWS IoT Core

Odczyt danych z czujnika oraz aktualizacje firmware przez sieć (OTA) · ESP32-S3 + ESP-IDF 5.5 · region eu-central-1

Notatka opisuje dwie rzeczy: **(A)** jak ustawić AWS tak, żeby ESP32 mógł wysyłać do chmury odczyty z czujnika, oraz **(B)** jak dołożyć do tego aktualizacje firmware przez sieć. Kroki po stronie AWS są opisane tak, jak wygląda to w konsoli przeglądarkowej. Fragmenty kodu pokazują tylko te miejsca, które łączą się z chmurą.

W przykładach używane są nazwy: urządzenie (Thing) `esp32s3-htu21d`, bucket `htu21d-ota-bucket`, region `eu-central-1` (Frankfurt). `<ACCOUNT_ID>` to 12-cyfrowy numer konta AWS, widoczny po kliknięciu nazwy konta w prawym górnym rogu konsoli.

Słowniczek

Pojęcie	Co to jest
AWS IoT Core	Usługa Amazona, która działa jako broker MQTT. To do niej łączy się urządzenie i przez nią przechodzą wszystkie dane.
Thing	Wpis w AWS reprezentujący jedno urządzenie. Ma nazwę, po której AWS je rozpoznaje.
Certyfikat urządzenia	Para plików (certyfikat + klucz prywatny), którą urządzenie udowadnia, że jest sobą. Odpowiednik loginu i hasła.
Polityka (policy)	Lista rzeczy, które urządzenie może robić: z czym się połączyć, na jakie tematy pisać i czego słuchać.
Temat (topic)	Adres wiadomości MQTT, np. <code>sensors/esp32s3-htu21d/data</code> .
S3 / bucket	Magazyn plików w chmurze. Bucket to pojedynczy folder w tym magazynie.
Rola IAM	Zestaw uprawnień, które jedna usługa AWS może tymczasowo przejąć, żeby coś zrobić w imieniu użytkownika.
Job (zadanie)	Polecenie wysłane do urządzenia, np. „zainstaluj wersję 1.0.1”. Urządzenie odbiera je przez MQTT i odsyła status.
Presigned URL	Podpisany link do prywatnego pliku w S3, ważny przez określony czas. Każdy, kto go ma, może pobrać ten jeden plik.

Część A · Odczyt danych z ESP32

Efekt końcowy: ESP32 budzi się, łączy z WiFi, odczytuje czujnik i wysyła wynik do AWS jako wiadomość MQTT, np. `{"t":23.45,"h":58.20}`.

A1. Wybór regionu

Wszystko trzeba robić w jednym regionie. Region wybiera się z listy w prawym górnym rogu konsoli, obok nazwy konta. W tej notatce jest to Europe (Frankfurt) `eu-central-1`.

Regionu nie da się później zmienić bez tworzenia wszystkiego od nowa, bo urządzenie, certyfikaty i adres brokera są przypisane do regionu.

A2. Utworzenie urządzenia (Thing)

1. W wyszukiwarce konsoli wpisz **IoT Core** i otwórz usługę.
2. W menu po lewej: **All devices** → **Things** → **Create things**.
3. Wybierz **Create single thing**.
4. W polu **Thing name** wpisz nazwę urządzenia, np. `esp32s3-htu21d`. Reszty pól nie trzeba wypełniać (Device Shadow zostaw na *No shadow*).

Ważne: zapamiętaj tę nazwę. Musi być identyczna z identyfikatorem klienta (Client ID) w kodzie, a przy aktualizacjach OTA wchodzi jeszcze w adresy tematów. Rozjazd nazw to najczęstsza przyczyna zrywanego połączenia.

A3. Certyfikat urządzenia

Na kolejnym ekranie kreatora wybierz **Auto-generate a new certificate**. Po utworzeniu AWS pokaże pliki do pobrania:

Plik	Do czego służy
Device certificate (<code>.pem.crt</code>)	Przedstawia urządzenie przed AWS.
Private key (<code>.pem.key</code>)	Dowód, że urządzenie jest właścicielem certyfikatu.
Amazon Root CA 1	Służy urządzeniu do sprawdzenia, czy łączy się z prawdziwym AWS.

Uwaga: klucza prywatnego nie da się pobrać drugi raz. Jeśli zginie, trzeba wygenerować nowy certyfikat. Kluczy nie wolno wrzucać do repozytorium — folder z nimi dodaj do `.gitignore`.

Na końcu kreatora certyfikat musi zostać **aktywowany** i przypięty do urządzenia. Sprawdzisz to w **Security** → **Certificates** — status ma być *Active*.

A4. Polityka

Bez polityki AWS zerwie połączenie od razu po zalogowaniu. Przejdź do **Security** → **Policies** → **Create policy**, nadaj nazwę i przełącz edytor na **JSON**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:client/esp32s3-htu21d"
    },
    {
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topic/sensors/esp32s3-htu21d/data"
    }
  ]
}
```

```
]
}
```

Co to znaczy:

- `iot:Connect` — pozwolenie na zalogowanie się do brokera pod tym identyfikatorem klienta.
- `iot:Publish` — pozwolenie na wysyłanie wiadomości na ten jeden temat.
- Gdyby urządzenie miało też odbierać wiadomości, dochodzą `iot:Subscribe` i `iot:Receive`. Przy `Subscribe` w adresie zasobu wpisuje się `topicfilter/` zamiast `topic/` — AWS tak rozróżnia oba przypadki.

Po zapisaniu politykę trzeba przypiąć do certyfikatu: **Security** → **Certificates** → (wybierz certyfikat) → **Attach policy**.

A5. Adres brokera (endpoint)

W IoT Core wejdź w **Settings** i skopiuj **Device data endpoint**. Wygląda tak:

```
xxxxxxxxxxxxx-ats.iot.eu-central-1.amazonaws.com
```

To adres, pod który łączy się urządzenie. Jest inny dla każdego konta.

A6. Ustawienia po stronie ESP32

Trzy pobrane pliki wrzuca się do projektu (np. `main/Certs/`) i wkompilewuje w program. W `main/CMakeLists.txt`:

```
idf_component_register(SRCS "main.c" "WiFi/wifi.c" "Sensor/sensor.c" "MQTT/mqtt.c"
                        INCLUDE_DIRS "." "Sensor"
                        EMBED_TXTFILES "Certs/AmazonRootCA1.pem"
                                      "Certs/device.pem.crt"
                                      "Certs/private.pem.key")
```

Dane dostępowe trzyma się w osobnym pliku (`secrets.h`) wykluczonym z repozytorium:

```
#define WIFI_SSID          "..."  
#define WIFI_PASS         "..."  
#define AWS_IOT_ENDPOINT  "xxxxxxxxxxxxx-ats.iot.eu-central-1.amazonaws.com"  
#define AWS_IOT_CLIENT_ID "esp32s3-htu21d" // musi być równe nazwie Thing
```

Konfiguracja klienta MQTT:

```
const esp_mqtt_client_config_t cfg = {  
    .broker.address.uri = "mqtt:// " AWS_IOT_ENDPOINT ":8883",  
    .broker.verificatio.certificate = aws_root_ca_pem,           // sprawdza AWS  
    .credentials.client_id = AWS_IOT_CLIENT_ID,                 // musi pasować do polityki  
    .credentials.authentication.certificate = device_cert_pem,  
    .credentials.authentication.key = private_key_pem,  
};
```

`mqtt://` i port 8883 oznaczają połączenie szyfrowane, w którym obie strony się sprawdzają: AWS weryfikuje certyfikat urządzenia, a urządzenie weryfikuje AWS przy pomocy Amazon Root CA.

A7. Sprawdzenie, czy dane docierają

1. W IoT Core otwórz **Test** → **MQTT test client**.

2. W zakładce **Subscribe to a topic** wpisz temat, np. `sensors/esp32s3-htu21d/data` (albo `#`, żeby widzieć wszystko).
3. Uruchom urządzenie. Wiadomości powinny pojawiać się na liście.

Typowe problemy

Objaw	Przyczyna
Połączenie zrywa się od razu, bez komunikatu błędu	Polityka. AWS przy braku uprawnień po prostu zamyka połączenie. Sprawdź, czy Client ID i temat zgadzają się z polityką co do znaku.
Błąd TLS / handshake	Zły certyfikat, nieaktywny certyfikat albo brak Amazon Root CA po stronie urządzenia.
Brak połączenia, timeout	Zły endpoint albo endpoint z innego regionu.

A8. Co dalej z danymi (opcjonalnie)

Same wiadomości MQTT nigdzie się nie zapisują. Żeby coś z nimi zrobić, tworzy się regułę: **Message routing → Rules → Create rule**. Reguła wybiera wiadomości zapytaniem SQL, np.:

```
SELECT t AS temperature, h AS humidity FROM 'sensors/esp32s3-htu21d/data'
```

i przekazuje je dalej — na przykład do CloudWatch (wykresy na dashboardzie), do bazy danych albo do funkcji Lambda.

Część B • Aktualizacje OTA

Zasada działania: nowy firmware leży w prywatnym buckecie S3. Zadanie (job) w AWS IoT mówi urządzeniu, jaką wersję ma zainstalować i skąd ją pobrać. Urządzenie pobiera plik przez HTTPS, instaluje go w drugiej partycji i restartuje się. Jeśli nowa wersja nie działa, wraca do poprzedniej.

B0. Czego wymaga urządzenie

To warunki wstępne po stronie ESP32 — bez nich konfiguracja AWS nic nie da.

Element	Ustawienie
Rozmiar flasha	W <code>menuconfig</code> musi odpowiadać rzeczywistości. Odczyt z układu: <code>esptool.py -p COM7 flash_id</code> (linia <i>Detected flash size</i>).
Tablica partycji	Dwie partycje aplikacji plus <code>otadata</code> . Własny plik <code>partitions.csv</code> w głównym folderze projektu, wybrany w <code>menuconfig</code> jako <i>Custom partition table</i> CSV.
Rollback	<code>CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE</code> włączony. Bez tego funkcje powrotu do poprzedniej wersji nic nie robią.
Numer wersji	<code>set(PROJECT_VER "1.0.0")</code> w głównym <code>CMakeLists.txt</code> , przed <code>include(...)</code> . Po tym numerze urządzenie rozpoznaje, czy zadanie dotyczy nowszej wersji.
Stos głównego zadania	Ok. 8192 bajtów. Pobieranie przez HTTPS nie mieści się w domyślnych 3584 i kończy się zawieszeniem.

Przykładowy plik `partitions.csv` (flash 16 MB):

```
# Name, Type, SubType, Offset, Size
nvs, data, nvs, , 0x6000
otadata, data, ota, , 0x2000
phy_init, data, phy, , 0x1000
ota_0, app, ota_0, , 2M
ota_1, app, ota_1, , 2M
```

Nowy program trafia zawsze do wolnej partycji, więc stara wersja zostaje nietknięta. `otadata` przechowuje informację, z której partycji startować. `nvs` służy m.in. do zapamiętania wersji, która okazała się wadliwa.

W kodzie pobieraniem zajmuje się `esp_https_ota`. Dwie rzeczy różnią konfigurację od zwykłego serwera w sieci lokalnej:

```
esp_http_client_config_t http_config = {
    .url = url, // link przychodzi z zadania
    .crt_bundle_attach = esp_crt_bundle_attach, // wbudowany zestaw certyfikatów (S3)
    .buffer_size_tx = 4096, // link ma 1-2 kB, domyślne 512 to za mało
};
```

`esp_crt_bundle_attach` podcina zestaw kilkuset certyfikatów urzędów certyfikacji wkompiłowany w program (opcja `CONFIG_MBEDTLS_CERTIFICATE_BUNDLE`). Dzięki temu urządzenie zweryfikuje S3 bez dokładania własnych plików.

B1. Rozszerzenie polityki o tematy zadań

Urządzenie rozmawia o zadaniach na zarezerwowanych tematach `$aws/things/<nazwa>/jobs/...`. Bez uprawnień AWS zerwie połączenie. W **Security** → **Policies** otwórz politykę, kliknij **Edit active version**, przełącz na JSON i dopisz dwa bloki:

```
{
  "Effect": "Allow",
  "Action": ["iot:Publish", "iot:Receive"],
  "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topic/$aws/things/esp32s3-htu21d/jobs/*"
},
{
  "Effect": "Allow",
  "Action": "iot:Subscribe",
  "Resource": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:topicfilter/$aws/things/esp32s3-htu21d/jobs/*"
}
```

Przy zapisie zaznacz **Set the edited version as the active version**.

B2. Bucket S3 na firmware

1. W konsoli otwórz **S3** → **Create bucket**.
2. **Region**: ten sam co IoT Core.
3. **Bucket name**: tylko małe litery, cyfry i myślniki, np. `htu21d-ota-bucket`. Nazwa musi być unikalna w skali świata, bo jest częścią adresu.
4. **Block all public access**: zostaw zaznaczone. Firmware nie powinien być dostępny publicznie — urządzenie dostanie podpisany link.

5. **Default encryption:** zostaw SSE-S3. Przy szyfrowaniu KMS rola z kroku B3 potrzebowałaby dodatkowych uprawnień do kluczy.

B3. Rola IAM do podpisywania linków

Bucket jest prywatny, a urządzenie nie ma żadnych uprawnień w S3 — ma tylko certyfikat do IoT Core. Dlatego link do pliku przygotowuje AWS IoT, korzystając z roli, która ma prawo czytać z bucketu.

Część 1: utworzenie roli

1. IAM → Roles → Create role.

2. **Trusted entity type:** *Custom trust policy*. Wklej:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "iot.amazonaws.com" },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": { "aws:SourceAccount": "<ACCOUNT_ID>" },
        "ArnLike": { "aws:SourceArn": "arn:aws:iot:eu-central-1:<ACCOUNT_ID>:*" }
      }
    }
  ]
}
```

Ten dokument mówi, kto może używać roli: usługa AWS IoT, ale tylko z Twojego konta i z tego regionu. Warunki są istotne, bo z usługi IoT korzystają wszyscy klienci AWS — bez nich ktoś obcy mógłby wskazać Twoją rolę w swoim zadaniu.

Na ekranie uprawnień nic nie zaznaczaj, nadaj nazwę (np. `iot-ota-s3-presign-role`) i utwórz rolę.

Część 2: uprawnienie do bucketu

Otwórz utworzoną rolę, **Add permissions** → **Create inline policy**, tryb JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::htu21d-ota-bucket/*"
    }
  ]
}
```

Rola może więc tylko pobierać pliki i tylko z tego jednego bucketu.

Co z czym jest powiązane: rola nie jest podpięta do bucketu — bucket nic o niej nie wie. Rola ma bucket wpisany w swoich uprawnieniach, a wybiera się ją przy tworzeniu zadania. Ta sama rola obsługuje wszystkie przyszłe aktualizacje, dopóki pliki leżą w tym samym buckecie.

B4. Pliki wydania

Jedno wydanie to dwa pliki. Firmware:

1. Zmień numer wersji w `CMakeLists.txt`, np. `set(PROJECT_VER "1.0.1")`.
2. Zbuduj: `idf.py build`.
3. Skopiuj `build/<nazwa_projektu>.bin` pod nazwą z numerem wersji, np. `firmware-1.0.1.bin`.

Oraz plik opisu zadania, np. `job-1.0.1.json`:

```
{
  "version": "1.0.1",
  "url": "${aws:iot:s3-presigned-url:https://s3.eu-central-1.amazonaws.com/htu21d-ota-bucket/firmware-1.0.1.bin}"
}
```

To zwykły plik tekstowy, pisany ręcznie. Jego treść ustalasz sam — urządzenie musi tylko umieć ją odczytać. Zapis `${aws:iot:s3-presigned-url:...}` to miejsce, w które AWS wstawi świeży podpisany link w chwili, gdy urządzenie poprosi o zadanie.

Oba pliki wgraj do bucketu: **S3** → (bucket) → **Upload** → **Add files**.

B5. Utworzenie zadania

1. **IoT Core** → **Remote actions** → **Jobs** → **Create job**.
2. Wybierz **Create a custom job**.
3. **Name**: np. `ota-1-0-1`. Kropki nie są dozwolone, stąd myślniki.
4. **Job targets** → **Things**: wybierz swoje urządzenie.
5. **Job document**: *From file* i wskaż `job-1.0.1.json` w buckecie.
6. Zaznacz **Pre-sign resource URLs**. AWS powinien wykryć adres pliku firmware i pokazać go pod spodem.
7. **Pre-signing role**: wybierz rolę z kroku B3.
8. **URL expiration**: domyślne 60 minut wystarcza z zapasem — pobranie 1 MB trwa kilkanaście sekund.
9. **Job configuration**: *Snapshot job*, czyli zadanie jednorazowe dla wskazanych urządzeń.
10. **Submit**. Od tej chwili zadanie jest aktywne i czeka, aż urządzenie o nie zapyta.

Uwaga: treść zadania jest kopiowana z pliku JSON w momencie tworzenia. Późniejsza zmiana pliku w buckecie nie wpływa na istniejące zadanie.

B6. Jak przebiega aktualizacja

1. Urządzenie budzi się, łączy z WiFi i AWS, wysyła odczyt z czujnika.
2. Pyta o zadanie: wysyła wiadomość na temat `$aws/things/<nazwa>/jobs/start-next`.
3. AWS przygotowuje podpisany link (używając roli) i odsyła treść zadania na temat `.../start-next/accepted`: identyfikator zadania, numer wersji i link.
4. Urządzenie porównuje numer wersji z własnym. Jeśli jest inny, pobiera plik z linku do wolnej partycji i restartuje się.
5. Po restarcie startuje nowa wersja i próbuje połączyć się z AWS. Udań połączenie jest dowodem, że wersja jest sprawna — dopiero wtedy zostaje zatwierdzona na stałe.

6. Przy kolejnym pytaniu o zadanie urządzenie widzi, że ma już wersję z zadania, i zgłasza `SUCCEEDED` na temat `.../jobs/<jobId>/update`. Zadanie jest zakończone.

Postęp widać w konsoli: **Jobs** → (zadanie) → **Job executions**, status zmienia się z *Queued* przez *In progress* na *Succeeded*. Strona nie odświeża się sama.

Gdy coś pójdzie nie tak

Sytuacja	Zachowanie urządzenia
Nowa wersja nie łączy się z AWS	Bootloader wraca do poprzedniej wersji, która czeka nietknięta w drugiej partycji. Wadliwa wersja trafia na czarną listę w pamięci NVS, a zadanie dostaje status <code>FAILED</code> .
Pobieranie przerwane (sieć, wygasły link)	Zadanie zostaje otwarte (<code>IN_PROGRESS</code>). Przy następnym wybudzeniu urządzenie spróbuje jeszcze raz, z nowym linkiem.
Wersja w pliku firmware nie zgadza się z wersją w zadaniu	Urządzenie zgłasza <code>FAILED</code> — do bucketu trafił plik z innym numerem wersji, niż mówi zadanie.

Ważne: wersja, która raz została odrzucona, jest zapamiętana i nie zostanie zainstalowana ponownie. Poprawkę trzeba wydać pod nowym numerem, np. 1.0.2 zamiast poprawionego 1.0.1.

B7. Wydanie kolejnej wersji

Zadania nie da się użyć drugi raz: po zgłoszeniu `SUCCEEDED` jest zamknięte i urządzenie już go nie dostanie. Treści istniejącego zadania też nie można podmienić. Każde wydanie to więc nowe zadanie:

- Nowy numer wersji w `CMakeLists.txt` i `idf.py build`.
- Kopia pliku JSON z podbitym numerem wersji w obu miejscach (pole `version` i nazwa pliku firmware).
- Wgranie obu plików do bucketu.
- Nowe zadanie w konsoli, np. `ota-1-0-2`.

Starych zadań nie trzeba kasować — zostają jako historia aktualizacji. Klikanie można skrócić: przycisk **Save as a job template** na stronie zadania zapamiętuje ustawienia (rolę, typ zadania), a przy pełnej automatyzacji to samo robi jedna komenda AWS CLI `aws iot create-job` w skrypcie razem z wysyłką pliku do S3.

B8. Szybka lista kontrolna

Objaw	Gdzie szukać
Zadanie stoi w <i>Queued</i>	Urządzenie w ogóle nie pyta o zadania: nie działa, nie ma WiFi albo polityka nie obejmuje tematów <code>jobs/*</code> .
Połączenie zrywa się po dodaniu obsługi zadań	Polityka: brak <code>topicfilter/</code> przy <code>Subscribe</code> albo nazwa Thing niezgodna z Client ID.
Pobieranie kończy się błędem HTTP 403	Rola nie ma uprawnień <code>s3:GetObject</code> do tego bucketu albo w zadaniu nieznaczono <i>Pre-sign resource URLs</i> .
Błąd zaraz po starcie pobierania	Za mały bufor na link (<code>buffer_size_tx</code>) lub brak zestawu certyfikatów.
Urządzenie restartuje się w kółko	Za mały stos głównego zadania albo nowa wersja nie potrafi połączyć się z AWS (wtedy <code>rollback zadziała sam</code>).

Załącznik: tematy MQTT używane przy aktualizacjach

Temat	Rola
<code>\$aws/things/<nazwa>/jobs/start-next</code>	Urządzenie pyta o następne zadanie.
<code>\$aws/things/<nazwa>/jobs/start-next/accepted</code>	Odpowiedź AWS z treścią zadania i linkiem.
<code>\$aws/things/<nazwa>/jobs/start-next/rejected</code>	Odpowiedź odmowna (np. błąd w treści zadania).
<code>\$aws/things/<nazwa>/jobs/<jobId>/update</code>	Urządzenie zgłasza status: <code>SUCCEEDED</code> lub <code>FAILED</code> .

Źródła: dokumentacja AWS IoT (Managing jobs, Jobs device MQTT API, Create and manage jobs by using the console), Amazon S3 (Creating a general purpose bucket, naming rules), ESP-IDF 5.5 (Partition Tables, OTA / App rollback, ESP x509 Certificate Bundle, ESP HTTP Client, ESP-MQTT).